In Chapter 3, *Tuesday: Methods*, on page 45, you also learned that it's generally a good idea to redefine respond_to? (or respond_to_missing?) together with method_missing. For example, if I can call my_task.description, then I expect that my_task.respond_to?(:description) returns true. The ActiveRecord::Base#respond_to? method is an *Around Alias (134)* of the original respond_to?, and it also checks whether a method name matches the rules for attribute readers, writers, or queries. The overridden respond_to? uses a *Nil Guard (219)* to calculate those names only once, and store them in an @@dynamic_methods class variable.

I stopped short of showing you the code that accesses the database, such as read_attribute, write_attribute, and query_attribute. Apart from that, you've just looked at the entire implementation of attribute methods in Rails 1. By the time Rails 2 came out, however, this code had become more complex.

## Rails 2: Focus on Performance

Do you remember the explanation of method_missing in Chapter 3, *Tuesday: Methods*, on page 45? When you call a method that doesn't exist, Ruby walks up the chain of ancestors looking for the method. If it reaches BasicObject without finding the method, then it starts back at the bottom and calls method_missing. This means that, in general, calling a *Ghost Method (57)* is slower than calling a normal method, because Ruby has to walk up the entire chain of ancestors at least once.

In most concrete cases, this difference in performance between Ghost Methods and regular methods is negligible. In Rails, however, attribute methods are called very frequently. In Rails 1, each of those calls also had to walk up ActiveRecord::Base's extremely long chain of ancestors. As a result, performance suffered.

The authors of Rails could solve this performance problem by replacing Ghost Methods with *Dynamic Methods (51)*—using define_method to create read, write, and query accessors for all attributes, and getting rid of method_missing altogether. Interestingly, however, they went for a mixed solution, including *both* Ghost Methods and Dynamic Methods. Let's look at the result.

### Ghosts Incarnated

If you check the source code of Rails 2, you'll see that the code for attribute methods moved from ActiveRecord::Base itself to a separate ActiveRecord::AttributeMethods module, which is then included by Base. The original method_missing has also become complicated, so we will discuss it in two separate parts. Here is the first part: